

Yash Prakash Mishra

Backend Engineer — Node.js, NestJS, TypeScript

+91 84007 93384 · mishra.yash3443@gmail.com · linkedin.com/in/yash-prakash-mishra-34063016a · github.com/xcalibre1

Happy to relocate to Berlin for an on-site role · Fluent English

About me

I build backend services in Node.js and TypeScript, mostly with NestJS. Five years in, the thread running through my work is other people's money: payments across four providers, loan servicing for 10,000+ active loans, and a rewards platform that verifies users, screens them for fraud and pays them out without anyone approving anything by hand. That last one is probably the most relevant thing I've built for Almedia.

What I actually enjoy is integration work, particularly the ugly parts, where a third party times out halfway through a request and someone has to decide what happens next. Most of what I'm proud of is unglamorous: a webhook handler that stopped paging people at 3am, a migration that moved years of payment history with no downtime, and a queue dashboard I open-sourced because debugging jobs blind annoyed me. Cursor and Claude are part of how I work rather than a novelty.

What I work with

Core	TypeScript, Node.js, NestJS, Express.js, Fastify, Jest
APIs	REST, WebSockets (Socket.io), Server-Sent Events, OpenAPI/Swagger, JWT, webhooks, idempotent endpoints, request validation
Databases	MySQL, MongoDB, Redis, PostgreSQL, ClickHouse, Sequelize, Prisma. Data modelling, zero-downtime migrations, query tuning
Queues and events	Kafka, BullMQ, AWS SQS. Publish/subscribe pipelines, event deduplication, retries, reconciliation
Integrations	Stripe, Razorpay, Affirm, Jodo, Xoxoday payouts, identity verification, reCAPTCHA. Owned end to end, from first API call to production
Infrastructure	Docker, Kubernetes (working knowledge), GitHub Actions, Jenkins, AWS (EC2, S3, SQS, KMS, IAM)
Keeping it up	Circuit breakers, backoff retries, ELK, structured logging, distributed tracing, on-call
Architecture	Microservices, event-driven systems, multi-tenancy, the usual patterns (Strategy, Factory, Observer)
AI tooling	Cursor, Claude and Copilot daily, for implementation, review and refactors
Front end, when needed	React.js, Next.js, Redux

Experience

Novatr — Senior Software Engineer (joined as Backend Developer)

Aug 2023 – Present

Y Combinator-backed SaaS, running 24/7 on AWS. I lead a team of five engineers.

Mostly: NestJS, Node.js, TypeScript, PostgreSQL, MySQL, Redis, ClickHouse, Kafka, BullMQ, Socket.io, Jest, Docker, AWS

Rewards and payouts

- Built the referral and rewards platform end to end: identity verification, fraud prevention, audit trails, and automatic payouts through Xoxoday. Referral conversions went up **25%**, and nobody approves a reward by hand any more.
- Fraud and audit were the hard parts, not the payouts. Anything that hands out something with cash value gets gamed, so every reward carries a trail of who earned it, why, and what was checked before it went out.

Payments and external services

- Built the payments platform. Four providers (Stripe, Razorpay, Affirm, Jodo) sit behind one interface and get picked at runtime, so adding a fifth doesn't mean touching checkout. I ran each integration myself, from the first API call to production.
- Made every payment path idempotent, so a retry, a duplicated provider callback or a double-clicked button can't charge someone twice.
- Wrote the webhook handler all four providers report into. It deduplicates events, retries itself when something fails, and reconciles status over Kafka. Manual error resolution dropped **80%**, which mostly meant people stopped getting paged.
- Added circuit breakers and backoff retries so a slow provider degrades instead of taking checkout down with it.

Scale, data and real time — Novatr

- Set up Kafka pipelines for payments, user lifecycle events and transactional email, so services stopped calling each other directly.
- Moved years of payment history off a legacy system onto ClickHouse with no downtime. Node ETL, schema and integrity checks, and a rollback path I actually tested.
- Built a live quiz engine on Socket.io and Redis for cohort sessions. Plenty of concurrent users, low latency, and it degrades gracefully when connections drop rather than falling over.

Platform

- Turned a B2C product into a multi-tenant B2B one: data isolation per tenant, per-tenant configuration deciding which features each client got, and onboarding flows. Five enterprise clients onboarded, and I was the engineer they talked to.
- Central RBAC across five internal tools, with role inheritance, scoped permissions and an audit log. Access incidents down **60%**. Also one JWT service handling token issuance, rotation and validation across three surfaces.
- Cohort setup used to be slow and broke often, so I rebuilt the CMS behind it with proper content lifecycle and versioning. Creation got **85%** faster and the recurring on-call incidents went away.

OLX Autos — Software Engineer

May 2022 – May 2023

Used-car financing for Latin America. Lending, so the edge cases have consequences.

Mostly: Node.js, TypeScript, Express.js, PostgreSQL, MySQL, Sequelize, REST, Docker, AWS

- Loan servicing for **10,000+ active loans** in Chile. I built the foreclosure and manual repayment flows that replaced work the operations team had been doing by hand.
- Modelled the loan lifecycle as a state machine with guards on every transition: legal states only, explicit rejections, an audit record per change. Manual intervention dropped **40%**.
- Built the risk and credit scoring workflows for loan origination. Approvals got **30%** faster, and you could see why a decision went the way it did.
- Wrote a PDF library for generating invoices inside the disbursement flow, which gave the operations team back 10+ hours a week. I was also their point of contact, nine time zones away.

Lenskart — Software Engineer (Frontend)

Apr 2021 – Apr 2022

India's largest eyewear retailer. The project ended when the UAE market launched.

Mostly: React.js, Redux, TypeScript, Next.js, Node.js (BFF), Webpack

- Rebuilt the core shopping flows (home, cart, checkout, order tracking) in React and TypeScript for the UAE launch, and got it out **30%** faster than planned.
- Worked against a Node.js BFF and agreed the response contracts with the backend team. That is where I got more interested in the backend than the front end.

Open source

BullMQ Dashboard

2025

Next.js, TypeScript, Node.js, Redis, ioredis, BullMQ, Docker, Server-Sent Events · github.com/xcalibre1/bullmq-dashboard

- Bull Board shows you queue state but won't let you do much about it: no retrigger with a corrected payload, no bulk actions, no live view. I wanted those at 2am during an incident, so I built them.
- Live stats over Server-Sent Events, an editable JSON payload editor, a throughput chart drawn in plain SVG, and bulk retry, promote, pause and resume. A Redis singleton and a cached BullMQ queue instance keep the API routes cheap.
- Docker Compose for one-command self-hosting. Engineering teams around the world use it now, which still surprises me.

Education

B.Tech, Computer Science — Rajkiya Engineering College, Sonbhadra

Jul 2016 – Jun 2020

Other things

- Happy to relocate to Berlin. I've spent five years working with teams across India, Latin America and the Middle East, so I'm used to writing things down, but I'd genuinely like to be in a room with people again.
- What I'm looking for: a product with real traffic, a team that ships quickly, and integration problems worth solving.